

Supplementary Material S1

Input-file specification for CADEMAs-ML

Pavel Novoa-Hernández and David A. Pelta

1 Scope

CADEMAS-ML evaluates a collection of cases using multiple H2O MOJO models and a separately defined fuzzy context. An executable analysis bundle comprises four coordinated inputs: a case dataset, model metadata, one or more context specifications, and the MOJO archives themselves. This supplement defines the expected structure of each input and the consistency conditions that connect them. The specification corresponds to the software version described in the accompanying article.

The repository contains a complete bundle under `example_attrition/`. Those files are an illustrative implementation of the contract below; they are not part of the generic schema and should not be interpreted as default policy choices for other domains.

Table 1: Files that constitute an analysis bundle.

Input	Format	Role
Case dataset	CSV	Supplies one row per case and the variables consumed by predictive models and context rules.
Model metadata	JSON	Associates each MOJO filename with performance metrics, feature names, and display metadata.
Predictive models	H2O MOJO (.zip)	Produce a positive-class probability for every case.
Context specification	JSON	Defines atomic memberships, derived rules, and optional final logic for contextual alignment.

2 Case dataset

2.1 Required structure

The dataset contains one row per case. Its columns must cover the union of:

- predictors required internally by the uploaded MOJO models;
- features listed in model metadata for local perturbation explanations; and
- features referenced by atomic context rules, including features used in **when** conditions.

Additional columns are permitted. A column named `Case_ID` or `ID` may be supplied to identify cases. If neither is present, the application generates consecutive identifiers. Identifier

columns are retained for display but excluded from model inference. Comma, semicolon, and tab delimiters are detected automatically.

2.2 Data consistency

Numerical context rules require values that can be converted to numeric form. Categorical values must use the same spelling and capitalization as the categories recorded by the MOJO model or context map. Missing-value handling follows the behavior of the corresponding MOJO and membership implementation; consequently, an analysis bundle should define and document an explicit missing-data policy before operational use.

The dataset may include a known outcome for retrospective inspection, as in the bundled `attrition` example. Such a column is not used in prioritization unless it is explicitly consumed by a model or context rule.

3 Model metadata JSON

3.1 Top-level structure

The document is a JSON object whose keys are MOJO filenames. Every uploaded model that participates in the analysis must have one corresponding entry, and the key must match the uploaded filename exactly.

```
{
  "model_a.zip": {
    "department": "Perspective A",
    "features": ["feature_1", "feature_2"],
    "performance": {
      "auc": 0.91,
      "aucpr": 0.88
    }
  }
}
```

Table 2: Fields in each model-metadata entry.

Field	Status	Meaning
<code>performance</code>	Required	Object mapping metric names to finite non-negative numerical values. The metric selected in the interface is normalized across participating models to obtain w_k as described by the model-weight equation in the article.
<code>features</code>	Required for explanations	Array of dataset columns perturbed by the model-agnostic local explanation procedure. Prediction can still rely on the schema embedded in the MOJO, but an empty list prevents feature-level attribution for that model.
<code>department</code>	Optional	Human-readable model or organizational-perspective label used in displays. The filename stem is used when this field is absent.

Metric names are user-defined, but the selected metric must be present for every participating

model. Values should be on a common scale and have the same interpretation. Negative, non-finite, or incomparable values are not meaningful under the normalization in Equation (4) of the article. If all selected values are zero, the application falls back to equal weights.

4 MOJO model archives

Each predictive ADM unit is supplied as an H2O MOJO archive with a `.zip` extension. Models may consume different predictor subsets, but all participating models must:

- accept the uploaded case matrix after identifier columns are removed;
- return one prediction per case;
- expose compatible positive-class probability columns; and
- use the same substantive interpretation of the positive class.

The software uses `p1` when that output is available and otherwise uses the final probability column returned by H2O. Agreement on column position alone is insufficient: bundle authors must verify that the positive class has the same semantic meaning across models.

5 Context configuration JSON

5.1 Document-level fields

Table 3: Top-level context fields.

Field	Status	Meaning
<code>context_name</code>	Recommended	Human-readable label shown in the context selector; the filename is used as fallback.
<code>description</code>	Optional	Brief explanation shown in the interface.
<code>rules</code>	Required	Array of atomic membership rules over dataset features.
<code>derived_rules</code>	Optional	Array of rules that aggregate previously defined atomic or derived memberships.
<code>logic</code>	Optional	Declarative final logic tree used when no explicit aggregation override is supplied.

5.2 Atomic rules

Every atomic rule requires a unique **name**, a dataset **feature**, a supported **type**, and compatible **params**. The optional **full_name** and **note** fields support human-readable displays and documentation.

An atomic rule can be restricted through a **when** object:

```
"when": { "feature": "department", "equals": "Sales" }
```

or

```
"when": { "feature": "region", "in": ["North", "West"] }
```

Rows that do not satisfy the condition receive a zero membership for that rule.

Table 4: Supported atomic membership types.

Type	Parameters	Constraint
linear_increasing	[a,b]	Numerical breakpoints with $a < b$. Membership increases from 0 to 1.
linear_decreasing	[a,b]	Numerical breakpoints with $a < b$. Membership decreases from 1 to 0.
triangular	[a,b,c]	Numerical breakpoints with $a < b < c$.
trapezoidal	[a,b,c,d]	Numerical breakpoints with $a < b \leq c < d$.
categorical_map	Object	Maps each known category directly to a membership in $[0, 1]$. Unspecified categories receive 0.
categorical_set	Object of arrays	Maps set names to category arrays. A numerical map must be supplied when the rule is consumed by a derived rule.

5.3 Derived rules

A derived rule has a unique **name**, an operator **op**, and a non-empty **inputs** array. Supported operators are **AVERAGE**, **MIN**, **MAX**, **PRODUCT**, **AND**, and **OR**; **AND** and **OR** use the minimum and maximum, respectively.

```
{
  "name": "career_scalability",
  "full_name": "Career scalability",
  "op": "PRODUCT",
  "inputs": [
    { "rule": "years_company_relevant" },
    { "rule": "years_role_relevant" }
  ]
}
```

Inputs normally reference an existing atomic or earlier derived rule. When an input refers to a **categorical_set**, it must provide the numerical conversion:

```
{
  "rule": "jobrole_critical",
  "map": { "critical": 1.0, "important": 0.7, "standard": 0.3 }
}
```

5.4 Final logic and interface override

The optional **logic** object uses the same recursive form as a derived rule. A direct reference has the form `{"rule":"name"}`; a composite node contains **op** and **inputs**. A final node may additionally declare **aggregation**. When present, this field determines the numerical aggregation while **op** retains its linguistic description.

The Streamlit interface provides three explicit final aggregation settings: *average*, *minimum* (*strict*), and *product*. For an interactive run, the selected setting takes precedence over the JSON logic and is applied to all numeric atomic and derived memberships stored in the audit matrix. The JSON logic is evaluated by the context engine only when no explicit aggregation override

is supplied. Authors should therefore report both the context file and the interface aggregation setting needed to reproduce a result.

6 Cross-file validation checklist

Before an analysis bundle is shared or used operationally, the following conditions should be checked:

1. Every uploaded MOJO filename exactly matches one model-metadata key.
2. The selected performance metric exists and is finite and non-negative for every participating model.
3. Every model and context feature exists in the case dataset with a compatible type.
4. The positive class has the same substantive meaning across all models.
5. Atomic and derived rule names are unique.
6. Every rule reference resolves to an atomic rule or an earlier derived rule.
7. Membership parameters have the required length and ordering.
8. Categorical maps cover expected values or document the zero-membership fallback.
9. The interface aggregation setting is recorded with exported results.
10. The bundle records software, model, data, and policy versions needed for audit and reproduction.

7 Bundled example

The repository example can be loaded with:

- `example_attrition/data/cases_attrition.csv`;
- `example_attrition/model_definitions.json`;
- the archives under `example_attrition/models/`; and
- either or both JSON files under `example_attrition/context/`.

The complete digital-transformation context is also formalized in Appendix B of the article. Machine-readable JSON Schemas for model metadata and context configuration accompany this supplement. They express the intended input contract and can be used for pre-submission or continuous-integration validation of new bundles.